Batch 12 — POS PWA Waiter Smartphone App

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:13

Batch 12 — POS PWA Waiter Smartphone App

Meat Lovers CIMS powered by YohPal

12A.

apps/pos-pwa/src/features/auth/api/authApi.js

```
import apiClient from '../../../lib/apiClient'

export const authApi = {
  login: async (payload) => {
    const { data } = await apiClient.post('/auth/login', payload)
    return data.data
  },
}
```

12B.

apps/pos-pwa/src/features/menu/api/menuApi.js

```
import apiClient from '../../../lib/apiClient'

export const menuApi = {
  products: async () => {
    const { data } = await apiClient.get('/products')
    return data.data.products
  },
}
```

12C.

apps/pos-pwa/src/features/orders/api/ordersApi.js

```
import apiClient from '../../../lib/apiClient'

export const ordersApi = {
  create: async (payload) => {
    const { data } = await apiClient.post('/orders', payload)
    return data.data
  },

  list: async () => {
    const { data } = await apiClient.get('/orders')
    return data.data.orders
  },

  requestCancellation: async (orderId, payload) => {
    const { data } = await apiClient.post(`/orders/${orderId}/request-cancellation`, payload)
    return data
  },

  requestDiscount: async (orderId, payload) => {
    const { data } = await apiClient.post(`/orders/${orderId}/request-discount`, payload)
    return data
  },
}
```

12D.

apps/pos-pwa/src/context/CartContext.jsx

```
import React, { createContext, useContext, useMemo, useState } from 'react'
```

```
const CartContext = createContext(null)

export function CartProvider({ children }) {
  const [tableNumber, setTableNumber] = useState('T01')
  const [items, setItems] = useState([])

  function addItem(product) {
    setItems((current) => {
      const existing = current.find((item) => item.product_id === product.id)

      if (existing) {
        return current.map((item) =>
          item.product_id === product.id
            ? { ...item, quantity: item.quantity + 1, total_price: (item.quantity + 1) * item.unit_price }
            : item
        )
      }

      return [
        ...current,
        {
          product_id: product.id,
          product_name: product.product_name,
          quantity: 1,
          unit_price: Number(product.selling_price),
          total_price: Number(product.selling_price),
        },
      ]
    })
  }

  function removeItem(productId) {
    setItems((current) => current.filter((item) => item.product_id !== productId))
  }

  function clearCart() {
    setItems([])
  }

  const subtotal = useMemo(
    () => items.reduce((sum, item) => sum + Number(item.total_price), 0),
    [items]
  )

  return (
    <CartContext.Provider
      value={{
        tableNumber,
        setTableNumber,
        items,
        addItem,
        removeItem,
        clearCart,
        subtotal,
      }}
    >
      {children}
    </CartContext.Provider>
  )
}

export function useCart() {
  return useContext(CartContext)
}
```

12E. Update

apps/pos-pwa/src/app/providers.jsx

```
import React from 'react'
import { BrowserRouter } from 'react-router-dom'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { CartProvider } from '../context/CartContext'

const queryClient = new QueryClient()

export default function AppProviders({ children }) {
  return (
    <QueryClientProvider client={queryClient}>
      <CartProvider>
        <BrowserRouter>{children}</BrowserRouter>
      </CartProvider>
    </QueryClientProvider>
```

```
  )
}
```

12F.

apps/pos-pwa/src/components/common/MobileNav.jsx

```
import React from 'react'
import { Link } from 'react-router-dom'

export default function MobileNav() {
  return (
    <nav
      style={{
        position: 'fixed',
        bottom: 0,
        left: 0,
        right: 0,
        background: '#111827',
        color: '#fff',
        display: 'grid',
        gridTemplateColumns: 'repeat(5, 1fr)',
        gap: 4,
        padding: 10,
        zIndex: 20,
      }}
    >
      <Link to="/menu">Menu</Link>
      <Link to="/table">Table</Link>
      <Link to="/cart">Cart</Link>
      <Link to="/orders">Orders</Link>
      <Link to="/requests">Requests</Link>
    </nav>
  )
}
```

12G.

apps/pos-pwa/src/layouts/PosLayout.jsx

```
import React from 'react'
import { Outlet } from 'react-router-dom'
import MobileNav from '../components/common/MobileNav'

export default function PosLayout() {
  return (
    <div style={{ minHeight: '100vh', padding: 16, paddingBottom: 90, background: '#f8fafc' }}>
      <header style={{ marginBottom: 16 }}>
        <strong>Meat Lovers POS powered by YohPal</strong>
      </header>

      <Outlet />

      <MobileNav />
    </div>
  )
}
```

12H.

apps/pos-pwa/src/app/routes.jsx

```
import React from 'react'
import { Navigate, Route, Routes } from 'react-router-dom'
import PosLayout from '../layouts/PosLayout'
import LoginPage from '../features/auth/pages/LoginPage'
import MenuPage from '../features/menu/pages/MenuPage'
import TableSelectionPage from '../features/tables/pages/TableSelectionPage'
import CartPage from '../features/cart/pages/CartPage'
import OrderStatusPage from '../features/orders/pages/OrderStatusPage'
import RequestsPage from '../features/requests/pages/RequestsPage'
import NotFound from '../pages/NotFound'

export default function AppRoutes() {
  return (
    <Routes>
      <Route path="/login" element={<LoginPage />} />

      <Route element={<PosLayout />}>
        <Route path="/" element={<Navigate to="/menu" replace />} />
```

```
        <Route path="/menu" element={<MenuPage />} />
        <Route path="/table" element={<TableSelectionPage />} />
        <Route path="/cart" element={<CartPage />} />
        <Route path="/orders" element={<OrderStatusPage />} />
        <Route path="/requests" element={<RequestsPage />} />
      </Route>

      <Route path="*" element={<NotFound />} />
    </Routes>
  )
}
```

12I.

apps/pos-pwa/src/features/auth/pages/LoginPage.jsx

```
import React, { useState } from 'react'
import { authApi } from '../api/authApi'

export default function LoginPage() {
  const [form, setForm] = useState({ email: '', password: '' })
  const [error, setError] = useState('')

  async function submit(e) {
    e.preventDefault()

    try {
      setError('')
      const result = await authApi.login(form)
      localStorage.setItem('pos_token', result.token)
      localStorage.setItem('pos_user', JSON.stringify(result.user))
      window.location.href = '/menu'
    } catch (err) {
      setError('Invalid login credentials')
    }
  }

  return (
    <main style={{ padding: 20 }}>
      <h1>Waiter POS Login</h1>
      <p>Meat Lovers CIMS powered by YohPal</p>

      {error ? <p style={{ color: 'red' }}>{error}</p> : null}

      <form onSubmit={submit} style={{ display: 'grid', gap: 12 }}>
        <input
          placeholder="Email"
          value={form.email}
          onChange={(e) => setForm({ ...form, email: e.target.value })}
          style={inputStyle}
        />

        <input
          placeholder="Password"
          type="password"
          value={form.password}
          onChange={(e) => setForm({ ...form, password: e.target.value })}
          style={inputStyle}
        />

        <button style={buttonStyle}>Open POS</button>
      </form>
    </main>
  )
}

const inputStyle = {
  padding: 14,
  borderRadius: 12,
  border: '1px solid #d1d5db',
}

const buttonStyle = {
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

12J.

apps/pos-pwa/src/features/menu/pages/MenuPage.jsx

```
import React, { useMemo, useState } from 'react'
import { useQuery } from '@tanstack/react-query'
import { menuApi } from '../api/menuApi'
import { useCart } from '../../../context/CartContext'

export default function MenuPage() {
  const [category, setCategory] = useState('FOOD')
  const { addItem } = useCart()

  const { data, isLoading, error } = useQuery({
    queryKey: ['pos-menu'],
    queryFn: menuApi.products,
  })

  const products = useMemo(
    () => (data || []).filter((item) => item.product_category === category),
    [data, category]
  )

  return (
    <main>
      <h1>Menu</h1>

      <div style={{ display: 'flex', gap: 8, overflowX: 'auto', marginBottom: 16 }}>
        <button onClick={() => setCategory('FOOD')} style={tabStyle}>Food</button>
        <button onClick={() => setCategory('SOFT_DRINK')} style={tabStyle}>Soft Drinks</button>
        <button onClick={() => setCategory('ALCOHOLIC_DRINK')} style={tabStyle}>Alcohol</button>
      </div>

      {isLoading ? <p>Loading menu...</p> : null}
      {error ? <p>Could not load menu.</p> : null}

      <div style={{ display: 'grid', gap: 12 }}>
        {products.map((product) => (
          <div key={product.id} style={cardStyle}>
            <div>
              <strong>{product.product_name}</strong>
              <p>KES {product.selling_price}</p>
            </div>

            <button onClick={() => addItem(product)} style={buttonStyle}>
              Add
            </button>
          </div>
        ))}
      </div>
    </main>
  )
}

const tabStyle = {
  padding: '10px 14px',
  borderRadius: 999,
  border: '1px solid #111827',
  background: '#fff',
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
  display: 'flex',
  justifyContent: 'space-between',
  gap: 12,
}

const buttonStyle = {
  padding: '10px 14px',
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

12K.

apps/pos-pwa/src/features/tables/pages/TableSelectionPage.jsx

```
import React from 'react'
import { useCart } from '../../../context/CartContext'
```

```
const tables = ['T01', 'T02', 'T03', 'T04', 'T05', 'VIP01', 'BAR01', 'TAKEAWAY']

export default function TableSelectionPage() {
  const { tableNumber, setTableNumber } = useCart()

  return (
    <main>
      <h1>Select Table</h1>
      <p>Current table: <strong>{tableNumber}</strong></p>

      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(2, 1fr)', gap: 12 }}>
        {tables.map((table) => (
          <button
            key={table}
            onClick={() => setTableNumber(table)}
            style={{
              padding: 20,
              borderRadius: 16,
              border: tableNumber === table ? '2px solid #111827' : '1px solid #d1d5db',
              background: tableNumber === table ? '#111827' : '#fff',
              color: tableNumber === table ? '#fff' : '#111827',
            }}
          >
            {table}
          </button>
        ))}
      </div>
    </main>
  )
}
```

12L.

apps/pos-pwa/src/features/cart/pages/CartPage.jsx

```
import React, { useState } from 'react'
import { ordersApi } from '../../orders/api/ordersApi'
import { useCart } from '../../../context/CartContext'

export default function CartPage() {
  const { tableNumber, items, removeItem, clearCart, subtotal } = useCart()
  const [paymentMethod, setPaymentMethod] = useState('CASH')
  const [status, setStatus] = useState('')

  async function createOrder() {
    const user = JSON.parse(localStorage.getItem('pos_user') || '{}')

    if (!items.length) {
      setStatus('Cart is empty')
      return
    }

    try {
      setStatus('Creating order...')

      const result = await ordersApi.create({
        customer_id: 1,
        waiter_id: user.id,
        table_number: tableNumber,
        subtotal,
        discount_amount: 0,
        total_amount: subtotal,
        payment_method: paymentMethod,
        items,
      })

      setStatus(`Order created: ${result.order_number}. Payment pending via ${paymentMethod}.`)
      clearCart()
    } catch (error) {
      setStatus('Could not create order')
    }
  }

  return (
    <main>
      <h1>Cart</h1>
      <p>Table: <strong>{tableNumber}</strong></p>

      <div style={{ display: 'grid', gap: 10 }}>
        {items.map((item) => (
          <div key={item.product_id} style={cardStyle}>
            <div>
              <strong>{item.product_name}</strong>
              <p>{item.quantity} × KES {item.unit_price}</p>
```

```
            </div>
            <button onClick={() => removeItem(item.product_id)}>Remove</button>
          </div>
        ))}
      </div>

      <h2>Total: KES {subtotal}</h2>

      <select
        value={paymentMethod}
        onChange={(e) => setPaymentMethod(e.target.value)}
        style={inputStyle}
      >
        <option value="CASH">Cash Pending</option>
        <option value="MPESA">M-Pesa Pending</option>
      </select>

      <br />
      <br />

      <button onClick={createOrder} style={buttonStyle}>
        Create Order
      </button>

      {status ? <p>{status}</p> : null}
    </main>
  )
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
  display: 'flex',
  justifyContent: 'space-between',
}

const inputStyle = {
  width: '100%',
  padding: 14,
  borderRadius: 12,
  border: '1px solid #d1d5db',
}

const buttonStyle = {
  width: '100%',
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

12M.

apps/pos-pwa/src/features/orders/pages/OrderStatusPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import { ordersApi } from '../api/ordersApi'

export default function OrderStatusPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['pos-orders'],
    queryFn: ordersApi.list,
  })

  return (
    <main>
      <h1>Order Status</h1>

      {isLoading ? <p>Loading orders...</p> : null}
      {error ? <p>Could not load orders.</p> : null}

      <div style={{ display: 'grid', gap: 12 }}>
        {(data || []).map((order) => (
          <div key={order.id} style={cardStyle}>
            <strong>{order.order_number}</strong>
            <p>Table: {order.table_number}</p>
            <p>Status: {order.order_status}</p>
            <p>Total: KES {order.total_amount}</p>
          </div>
        ))}
```

```
      </div>
    </main>
  )
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
}
```

12N.

apps/pos-pwa/src/features/requests/pages/RequestsPage.jsx

```
import React, { useState } from 'react'
import { ordersApi } from '../../orders/api/ordersApi'

export default function RequestsPage() {
  const [cancelForm, setCancelForm] = useState({ order_id: '', reason: '' })
  const [discountForm, setDiscountForm] = useState({ order_id: '', reason: '', discount_amount: '' })
  const [status, setStatus] = useState('')

  async function requestCancel(e) {
    e.preventDefault()

    try {
      await ordersApi.requestCancellation(cancelForm.order_id, {
        reason: cancelForm.reason,
      })

      setStatus('Cancellation request submitted for approval.')
    } catch (error) {
      setStatus('Could not submit cancellation request.')
    }
  }

  async function requestDiscount(e) {
    e.preventDefault()

    try {
      await ordersApi.requestDiscount(discountForm.order_id, {
        reason: discountForm.reason,
        discount_amount: Number(discountForm.discount_amount),
      })

      setStatus('Discount request submitted for approval.')
    } catch (error) {
      setStatus('Could not submit discount request.')
    }
  }

  return (
    <main>
      <h1>Approval Requests</h1>

      {status ? <p>{status}</p> : null}

      <section style={cardStyle}>
        <h3>Cancel Order Request</h3>
        <form onSubmit={requestCancel} style={{ display: 'grid', gap: 10 }}>
          <input
            placeholder="Order ID"
            value={cancelForm.order_id}
            onChange={(e) => setCancelForm({ ...cancelForm, order_id: e.target.value })}
            style={inputStyle}
          />
          <textarea
            placeholder="Reason"
            value={cancelForm.reason}
            onChange={(e) => setCancelForm({ ...cancelForm, reason: e.target.value })}
            style={inputStyle}
          />
          <button style={buttonStyle}>Submit Cancellation</button>
        </form>
      </section>

      <section style={cardStyle}>
        <h3>Discount Request</h3>
        <form onSubmit={requestDiscount} style={{ display: 'grid', gap: 10 }}>
          <input
            placeholder="Order ID"
            value={discountForm.order_id}
```

```
            onChange={(e) => setDiscountForm({ ...discountForm, order_id: e.target.value })}
            style={inputStyle}
          />
          <input
            placeholder="Discount Amount"
            value={discountForm.discount_amount}
            onChange={(e) => setDiscountForm({ ...discountForm, discount_amount: e.target.value })}
            style={inputStyle}
          />
          <textarea
            placeholder="Reason"
            value={discountForm.reason}
            onChange={(e) => setDiscountForm({ ...discountForm, reason: e.target.value })}
            style={inputStyle}
          />
          <button style={buttonStyle}>Submit Discount</button>
        </form>
      </section>
    </main>
  )
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
  marginBottom: 16,
}

const inputStyle = {
  width: '100%',
  padding: 14,
  borderRadius: 12,
  border: '1px solid #d1d5db',
}

const buttonStyle = {
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}
```

12O.

apps/pos-pwa/src/pages/NotFound.jsx

```
import React from 'react'

export default function NotFound() {
  return (
    <main style={{ padding: 20 }}>
      <h1>Page not found</h1>
      <p>The POS page you are looking for does not exist.</p>
    </main>
  )
}
```

12P.

docs/POS_PWA_WAITER_APP.md

```
# Batch 12 — POS PWA Waiter Smartphone App

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch builds the mobile-first waiter POS app.

## Features Added

1. Waiter login
2. Mobile menu
3. Product category tabs
4. Table selection
5. Cart
6. Create order
```

```
7. Payment pending state
8. Order status view
9. Cancel request
10. Discount request
11. Mobile-first bottom navigation

## Product Separation

The menu separates:

- Food
- Soft drinks
- Alcoholic drinks

## Waiter Flow

1. Waiter logs in
2. Waiter selects table
3. Waiter opens menu
4. Waiter adds products to cart
5. Waiter selects payment state
6. Waiter creates order
7. Kitchen sees order
8. Cashier settles payment
9. Manager approves cancellations or discounts if requested

## Control Rule

Waiters cannot directly cancel or discount orders.

They can only submit:

- cancellation request
- discount request

Approval must be handled by manager/admin.
```

Batch 12 Outcome

POS PWA now includes:

✅ waiter login
✅ mobile menu
✅ table selection
✅ cart
✅ create order
✅ order status
✅ cash/M-Pesa pending payment state
✅ cancel request
✅ discount request
✅ mobile-first POS navigation

Next Smart Move

Build **Batch 13 — Kitchen + Bar Mobile Screens**, including:

- kitchen login-compatible screen
- kitchen order queue
- mark preparing
- mark ready
- bar stock screen
- bar issue screen
- bar low-stock alert
- mobile-first kitchen/bar navigation